LAPORAN TUGAS MLOPS PERTEMUAN 6

Hands-on 1: Model Experimentation & Tracking dengan MLflow

Nama : **Fairuz Ary Syifa**
NIM : **123450044**
Kelas : **RA**

Pendahuluan

Dalam pengembangan model *machine learning*, seorang praktisi biasanya menjalankan banyak percobaan dengan kombinasi parameter, data, dan algoritma yang berbeda. Tanpa pencatatan yang rapi, hasil percobaan mudah tercecer dan sulit direproduksi. **MLflow** adalah *platform open-source* yang membantu mencatat, membandingkan, dan mengelola percobaan serta model secara terpusat.

Laporan ini mendokumentasikan praktik Hands-on 1 pada modul Pertemuan 6 yang mencakup enam tahap, yaitu (A) logging dasar, (B) logging per *epoch* pada PyTorch, (C) *autologging*, (D) *hyperparameter tuning* dengan *nested runs* (Optuna + MLflow), (E) membandingkan dan mencari *run* secara programatik, serta (F) *Model Registry*. Seluruh percobaan menggunakan MLflow Tracking Server lokal pada http://localhost:5000 dan *dataset* Breast Cancer Wisconsin bawaan scikit-learn.

Ringkasan eksperimen yang dibuat selama praktik:

Tahap	Skrip	Nama eksperimen MLflow	Isi pencatatan
A	train_basic.py	breast-cancer-clf (ID 1)	Parameter, tag, metrik, dan model Random Forest
B	train_pytorch.py	pytorch-epoch-logging (ID 2)	Metrik per epoch, checkpoint, dan model PyTorch
C	train_autolog.py	breast-cancer-autolog (ID 3)	Pencatatan otomatis oleh autolog
D, F	train_optuna.py, train_and_register.py	breast-cancer-optuna (ID 4)	1 parent run dan 5 child run, serta run registrasi model
E	search_runs.py	breast-cancer-optuna	Pencarian dan perbandingan run

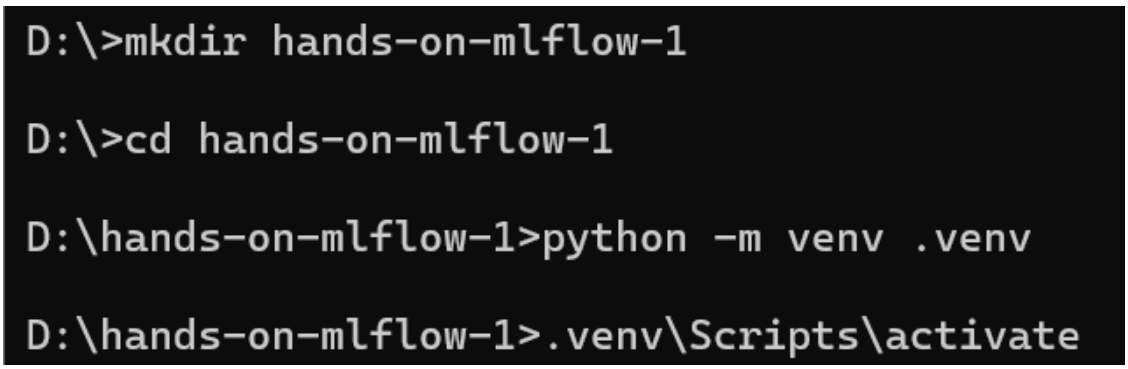
A. Logging Dasar

Tujuan

Memahami alur dasar MLflow Tracking, yaitu menjalankan server MLflow, menghubungkan skrip Python ke server tersebut, lalu mencatat parameter, tag, metrik, dan model dalam satu *run*.

1. Menyiapkan folder dan lingkungan kerja

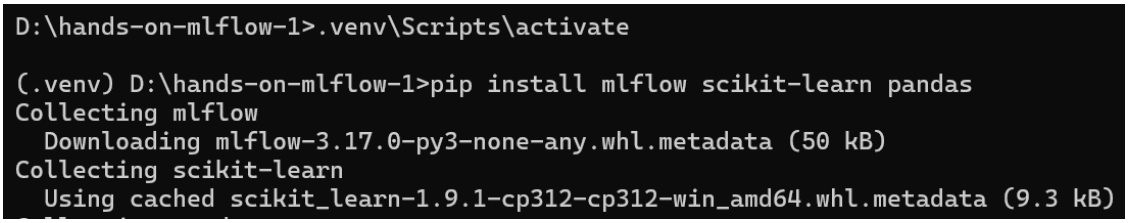
Langkah pertama adalah membuat folder proyek hands-on-mlflow-1, berpindah ke folder tersebut, lalu membuat dan mengaktifkan *virtual environment* (.venv). *Virtual environment* dipakai agar pustaka yang diinstal untuk praktik ini terisolasi dan tidak mengganggu instalasi Python lain di komputer.



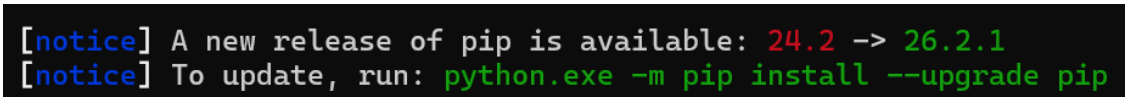
Gambar 1. Membuat folder proyek dan virtual environment

2. Instalasi MLflow dan scikit-learn

Setelah *virtual environment* aktif, dipasang pustaka mlflow (untuk pencatatan eksperimen), scikit-learn (untuk membangun model), dan pandas (dibutuhkan untuk menampilkan hasil pencarian *run* pada tahap E). Versi MLflow yang terpasang adalah 3.17.0. Pesan *notice* pada akhir proses hanya memberi tahu bahwa tersedia versi pip yang lebih baru dan tidak memengaruhi praktik.



Gambar 2. Instalasi mlflow, scikit-learn, dan pandas

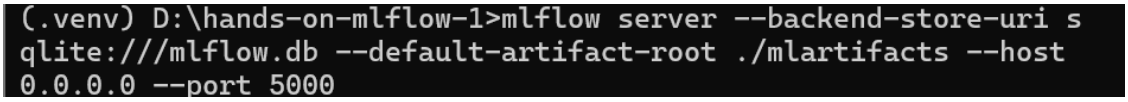


Gambar 3. Notifikasi pembaruan pip (informasi, bukan error)

3. Menjalankan server MLflow (Terminal 1)

Server MLflow dijalankan di terminal pertama dan dibiarkan terus aktif selama praktik. Opsi yang digunakan pada perintah:

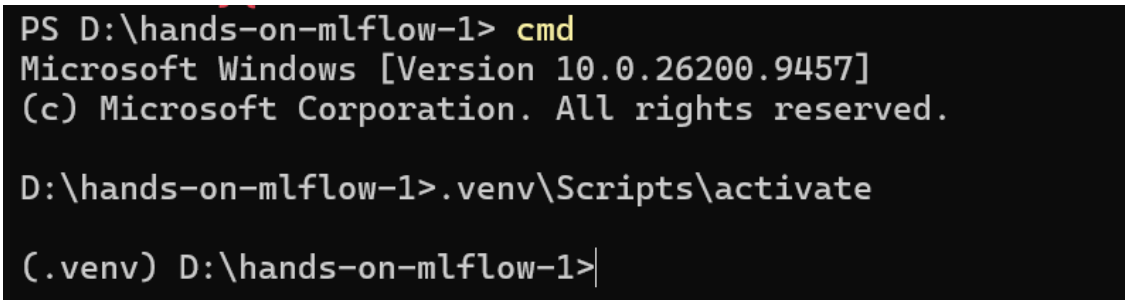
- --backend-store-uri sqlite:///mlflow.db menyimpan metadata (run, parameter, metrik) pada basis data SQLite lokal bernama mlflow.db.
- --default-artifact-root ./mlartifacts menyimpan *artifact* (model, file, gambar) pada folder mlartifacts.
- --host 0.0.0.0 --port 5000 membuka server pada port 5000 sehingga UI dapat diakses di http://localhost:5000.



Gambar 4. Perintah menjalankan MLflow server

4. Menyiapkan terminal kedua (Terminal 2)

Karena Terminal 1 sibuk menjalankan server, dibuka terminal baru untuk menjalankan skrip latihan. *Virtual environment* perlu diaktifkan kembali di terminal ini (ditandai dengan awalan (.venv) pada *prompt*).



Gambar 5. Terminal kedua dengan virtual environment aktif

5. Membuat skrip train_basic.py

Skrip train_basic.py dibuat di VS Code pada folder proyek, kemudian diisi dengan kode berikut.

```
# train_basic.py
import mlflow
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score, roc_auc_score
from sklearn.model_selection import train_test_split

# Hubungkan ke server MLflow lokal dan buat nama eksperimen
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("breast-cancer-clf")

# Siapkan data
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_val, y_tr, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Definisikan parameter model
params = {"n_estimators": 200, "max_depth": 8, "random_state": 42}

# Mulai proses pencatatan (Run) di MLflow
with mlflow.start_run(run_name="rf-baseline"):
  # Beri tag tambahan
  mlflow.set_tags({"stage": "baseline", "author": "pemula"})

  # Log parameter
  mlflow.log_params(params)

  # Latih model Random Forest
  model = RandomForestClassifier(**params).fit(X_tr, y_tr)
  proba = model.predict_proba(X_val)[:, 1]

  # Hitung dan log metrik
```

```
  mlflow.log_metric("val_f1", f1_score(y_val, proba > 0.5))
  mlflow.log_metric("val_auc", roc_auc_score(y_val, proba))

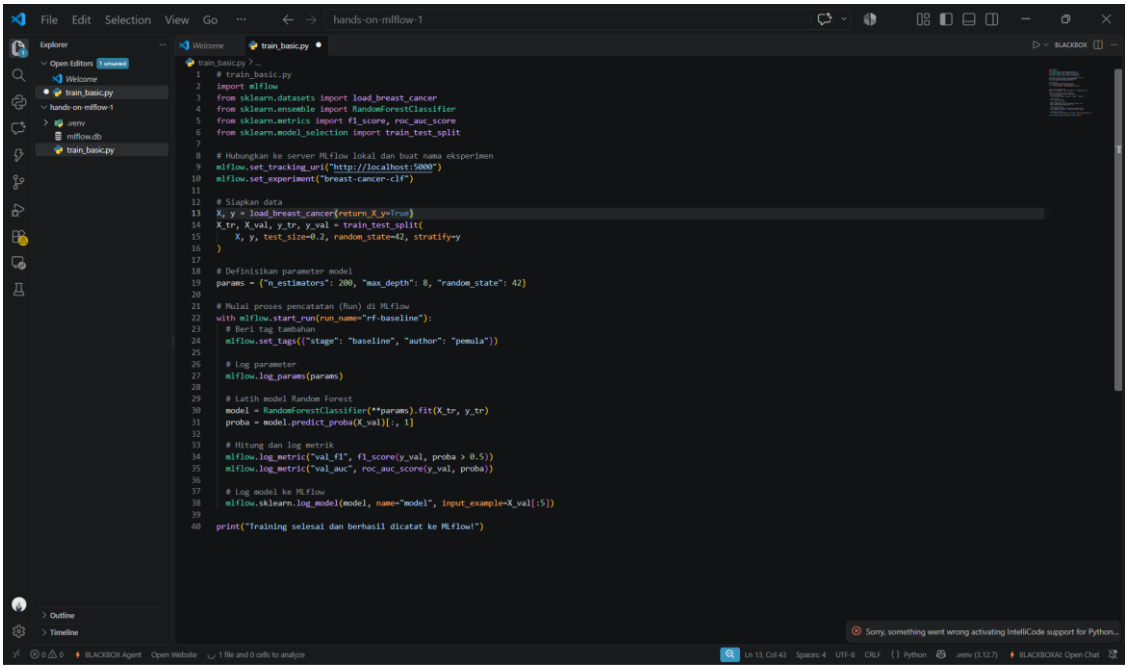
  # Ganti baris mlflow.sklearn.log_model dengan kode ini:
mlflow.sklearn.log_model(
    model,
    name="model",
    input_example=X_val[:5],
    skops_trusted_types=["sklearn.tree._tree.Tree"],
)



print("Training selesai dan berhasil dicatat ke MLflow!")
```

Penjelasan kode. Skrip melatih model *Random Forest* untuk mengklasifikasikan tumor payudara (jinak atau ganas) dan mencatat seluruh prosesnya ke MLflow:

- set_tracking_uri dan set_experiment menghubungkan skrip ke server lokal serta membuat (atau memakai) eksperimen bernama breast-cancer-clf sebagai wadah semua *run*.
- load_breast_cancer memuat dataset bawaan scikit-learn, lalu train_test_split membaginya menjadi 80% data latih dan 20% data validasi. Parameter stratify=y menjaga proporsi kelas tetap sama di kedua bagian dan random_state=42 membuat pembagian dapat direproduksi.
- params berisi hiperparameter model (200 pohon, kedalaman maksimum 8).
- mlflow.start_run(run_name="rf-baseline") membuka sebuah *run*; semua pencatatan di dalam blok with masuk ke *run* ini.
- set_tags memberi label bebas (stage=baseline, author=pemula) untuk memudahkan pencarian, log_params mencatat hiperparameter, dan log_metric mencatat metrik evaluasi val_f1 dan val_auc pada data validasi.
- mlflow.sklearn.log_model menyimpan model terlatih sebagai *artifact*. input_example menyimpan contoh input (5 baris data validasi) sehingga MLflow dapat menyimpulkan *signature* model, sedangkan skops_trusted_types mengizinkan tipe objek sklearn.tree._tree.Tree agar model dapat disimpan dengan format skops yang lebih aman.



Gambar 6. Kode train_basic.py di VS Code

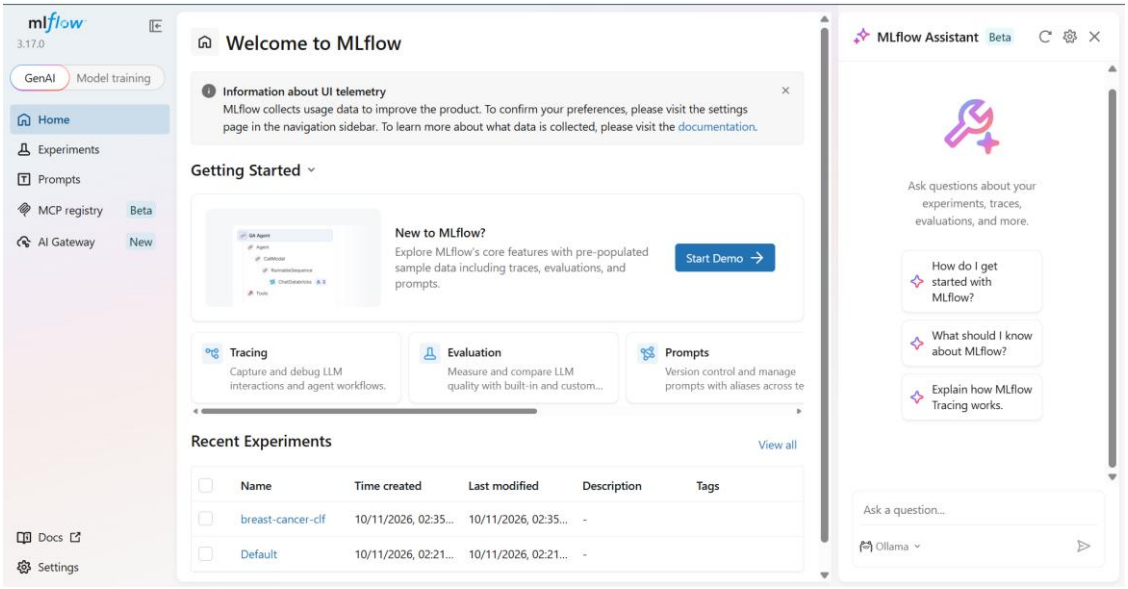
6. Menjalankan skrip dan memeriksa hasil

Skrip dijalankan di Terminal 2 dengan perintah python train_basic.py. Keluaran menampilkan tautan *run* rf-baseline pada eksperimen 1 dan pesan *"Training selesai dan berhasil dicatat ke MLflow!"* yang menandakan seluruh pencatatan berhasil.

```
(.venv) D:\hands-on-mlflow-1>python train_basic.py
🏃 View run rf-baseline at: http://localhost:5000/#/experiments/
1/runs/4fa59cae6935491f9be94878aa52d7a5
🧪 View experiment at: http://localhost:5000/#/experiments/1
Training selesai dan berhasil dicatat ke MLflow!
```

Gambar 7. Hasil menjalankan train_basic.py

Selanjutnya, UI MLflow dibuka melalui browser pada http://127.0.0.1:5000. Eksperimen breast-cancer-clf beserta *run* rf-baseline muncul di UI. Di sana dapat dilihat parameter, metrik, tag, dan model yang telah dicatat, sehingga percobaan ini terdokumentasi dan dapat dibuka kembali kapan saja.



Gambar 8. Tampilan MLflow UI di browser

B. Deep Learning: Logging per Epoch (PyTorch)

Tujuan

Mempraktikkan pencatatan metrik pada setiap *epoch* sehingga perkembangan pelatihan dapat dipantau sebagai kurva, serta menyimpan *checkpoint* model terbaik sebagai *artifact*.

1. Instalasi PyTorch

Dipasang pustaka torch dan torchvision pada *virtual environment* yang sama.

```
(.venv) D:\hands-on-mlflow-1>pip install torch torchvision
```

Gambar 9. Instalasi torch dan torchvision

```
[notice] A new release of pip is available: 24.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Gambar 10. Notifikasi pembaruan pip setelah instalasi

2. Membuat skrip train_pytorch.py

Skrip train_pytorch.py dibuat pada folder hands-on-mlflow-1 dengan kode berikut, mengikuti struktur pada modul.

```
# train_pytorch.py
import mlflow
import torch
import torch.nn as nn
import torch.optim as optim



# Fungsi dummy untuk simulasi training per epoch (sesuai alur modul)
def train_one_epoch(model, dataloader, criterion, optimizer):
  model.train()
  return 0.25
```

```
def evaluate(model, dataloader, criterion):
  model.eval()
  return 0.15, 0.88



# Konfigurasi parameter (cfg)
cfg = {
    "lr": 3e-4,
    "batch_size": 64,
    "epochs": 5,
    "arch": "resnet18",
    "seed": 42,
}

torch.manual_seed(cfg["seed"])

# Hubungkan ke server MLflow lokal
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("pytorch-epoch-logging")

# Definisikan model sederhana, loss, dan optimizer
model = nn.Sequential(nn.Linear(10, 16), nn.ReLU(), nn.Linear(16, 2))
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=cfg["lr"])

# Mulai MLflow Run
with mlflow.start_run(run_name=f"{cfg['arch']}-lr{cfg['lr']}" ):
  mlflow.log_params(cfg)
  mlflow.log_artifact(__file__)

  best_f1 = 0.0

  # Looping per epoch sesuai modul slide 44-45
  for epoch in range(cfg["epochs"]):
    train_loss = train_one_epoch(model, None, criterion, optimizer)
    val_loss, val_f1 = evaluate(model, None, criterion)

    # Log metrics per epoch
    mlflow.log_metrics(
        {"train_loss": train_loss, "val_loss": val_loss, "val_f1": val_f1},
        step=epoch,
    )

    if val_f1 > best_f1:
      best_f1 = val_f1
      torch.save(model.state_dict(), "best.pt")

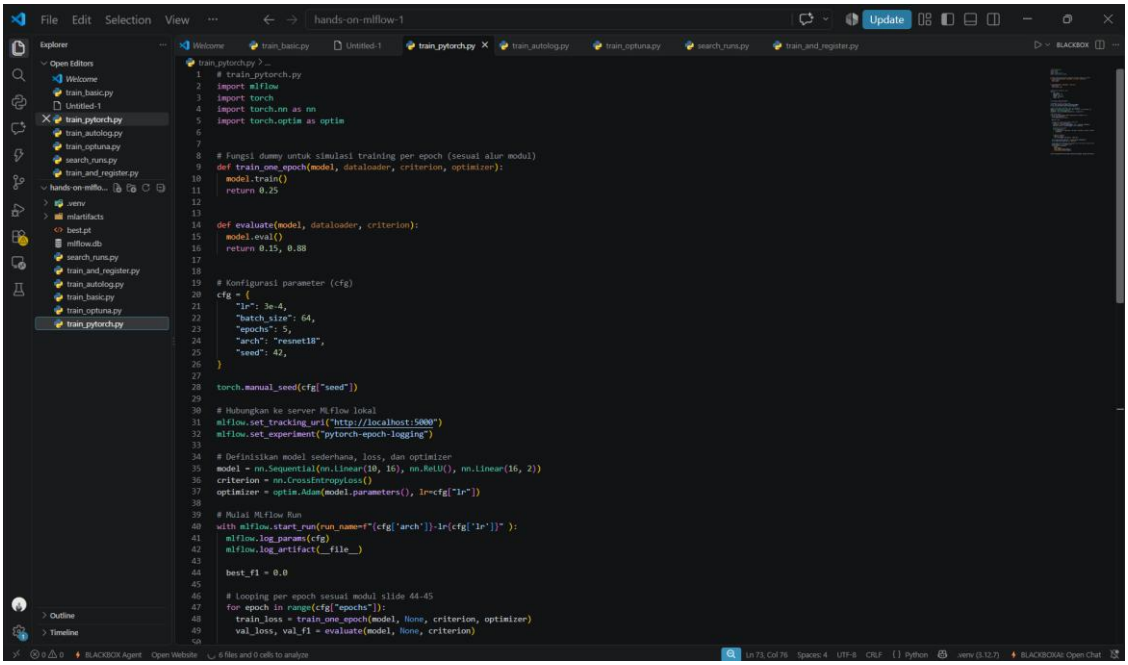
  mlflow.log_artifact("best.pt", artifact_path="checkpoints")
  mlflow.log_metric("best_val_f1", best_f1)

  # Log model PyTorch dengan input_example dan serialization format pickle
  input_example = torch.randn(1, 10)
  mlflow.pytorch.log_model(
      model,
      name="model",
      input_example=input_example,
      serialization_format="pickle",
  )

print("Training PyTorch per epoch selesai dan berhasil dicatat ke MLflow!")
```

Penjelasan kode.

- cfg menyimpan konfigurasi percobaan (laju belajar lr, batch_size, jumlah epochs, nama arsitektur, dan seed). torch.manual_seed mengunci *seed* agar hasil dapat direproduksi.
- Fungsi train_one_epoch dan evaluate adalah fungsi simulasi (*dummy*) yang mengembalikan nilai tetap (*train loss* 0,25, *val loss* 0,15, *val F1* 0,88). Dengan demikian, praktik ini berfokus pada mekanisme pencatatan MLflow, bukan pada proses pelatihan yang sebenarnya.
- Model yang dibuat adalah jaringan saraf sederhana (nn.Sequential) dengan *loss* CrossEntropyLoss dan *optimizer* Adam.
- Di dalam *run*, log_params(cfg) mencatat konfigurasi dan log_artifact(__file__) menyimpan salinan skrip itu sendiri agar kode yang menghasilkan *run* ini ikut terdokumentasi.
- Pada setiap *epoch*, log_metrics(..., step=epoch) mencatat train_loss, val_loss, dan val_f1 beserta nomor langkahnya. Parameter step inilah yang membuat MLflow dapat menampilkan grafik metrik terhadap *epoch*.
- Jika val_f1 lebih baik daripada sebelumnya, bobot model disimpan ke best.pt. Setelah perulangan selesai, file tersebut diunggah ke folder checkpoints sebagai *artifact*, dan nilai terbaik dicatat sebagai best_val_f1.
- mlflow.pytorch.log_model menyimpan model PyTorch utuh dengan input_example berupa tensor acak berukuran 1×10 dan format serialisasi pickle.



Gambar 11. Kode train_pytorch.py di VS Code

3. Menjalankan skrip

Skrip dijalankan dengan python train_pytorch.py. Keluaran menampilkan *run* resnet18-lr0.0003 pada eksperimen 2 (pytorch-epoch-logging) dan pesan keberhasilan. Peringatan (*WARNING*)

yang muncul berasal dari MLflow: format pickle dapat mengeksekusi kode sembarang saat dibaca ulang sehingga hanya aman dipakai pada model yang dipercaya. Peringatan ini bukan kesalahan dan proses pencatatan tetap berhasil.

```
(.venv) D:\hands-on-mlflow-1>python train_pytorch.py
2026/10/11 03:03:04 WARNING mlflow.pytorch: Saving pytorch model
 by Pickle or CloudPickle format requires exercising caution bec
ause these formats rely on Python's object serialization mechani
sm, which can execute arbitrary code during deserialization. The
 recommended safe alternative is to set `serialization_format` t
o 'pt2' to save the PyTorch model using the safe graph model for
mat.
🏃 View run resnet18-lr0.0003 at: http://localhost:5000/#/experi
ments/2/runs/4c1c3601b76b4f21a65860f1d8799dab
🧪 View experiment at: http://localhost:5000/#/experiments/2
Training PyTorch per epoch selesai dan berhasil dicatat ke MLflo
w!
```

Gambar 12. Hasil menjalankan train_pytorch.py

C. Autologging

Tujuan

Mencoba fitur *autologging*, yaitu pencatatan parameter, metrik, dan model secara otomatis hanya dengan satu baris kode, sehingga kode pencatatan manual tidak perlu ditulis berulang.

1. Membuat skrip train_autolog.py

```
# train_autolog.py
import mlflow
from sklearn.datasets import load  breast  cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# 1. Aktifkan autologging untuk scikit-learn
mlflow.sklearn.autolog()

# 2. Hubungkan ke server MLflow lokal dan buat eksperimen baru
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("breast-cancer-autolog")

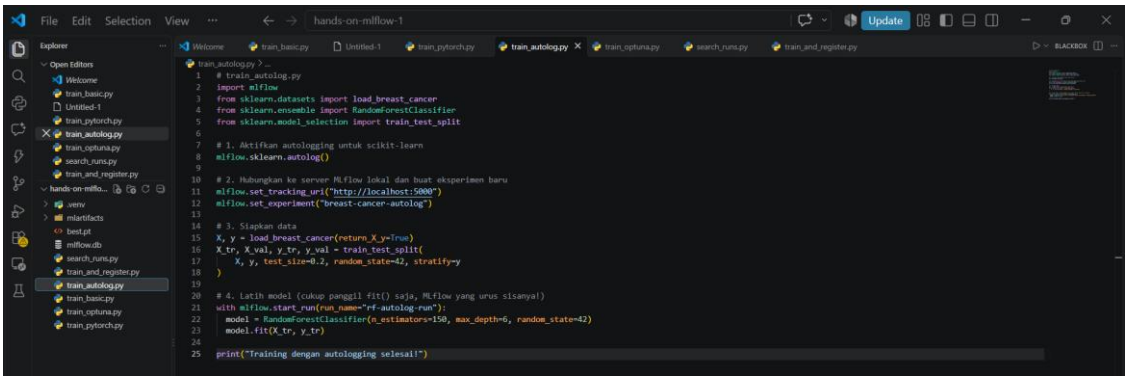
# 3. Siapkan data
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_val, y_tr, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 4. Latih model (cukup panggil fit() saja, MLflow yang urus sisanya!)
with mlflow.start_run(run_name="rf-autolog-run"):
  model = RandomForestClassifier(n_estimators=150, max_depth=6, random_state=42)
  model.fit(X_tr, y_tr)

print("Training dengan autologging selesai!")
```

Penjelasan kode.

- mlflow.sklearn.autolog() mengaktifkan pencatatan otomatis untuk scikit-learn. Setelah itu, setiap pemanggilan fit() otomatis mencatat seluruh hiperparameter model, metrik pada data latih, serta model terlatih lengkap dengan *signature*-nya.
- Eksperimen yang digunakan bernama breast-cancer-autolog, dengan data dan pembagian yang sama seperti tahap A agar hasilnya mudah dibandingkan.
- Model dilatih cukup dengan memanggil model.fit(...) di dalam start_run; tidak ada pemanggilan log_param, log_metric, maupun log_model secara manual.



Gambar 13. Kode train_autolog.py di VS Code

2. Menjalankan skrip

Perintah python train_autolog.py menghasilkan *run* rf-autolog-run pada eksperimen 3. Peringatan FutureWarning berasal dari scikit-learn versi 1.9 yang mengganti nama argumen y_pred menjadi y_proba pada fungsi evaluasi internal; ini hanya pemberitahuan perubahan versi dan tidak menggagalkan proses.

```
D:\hands-on-mlflow-1\.venv\Lib\site-packages\sklearn\metrics\_cl
assification.py:3424: FutureWarning: `y_pred` was renamed to `y_
proba` in version 1.9 and will be removed in 1.11. Use `y_proba`
 instead.
  warnings.warn(
🏃 View run rf-autolog-run at: http://localhost:5000/#/experimen
ts/3/runs/e6522eb3c9614e51887f4564a29b2d2d
🧪 View experiment at: http://localhost:5000/#/experiments/3
Training dengan autologging selesai!
```

Gambar 14. Hasil menjalankan train_autolog.py

Catatan. *Autologging* praktis dan konsisten antar *run*, tetapi tidak mengetahui versi data maupun konteks bisnis, sehingga informasi seperti versi data atau metrik khusus tetap perlu dicatat manual bila dibutuhkan.

D. Hyperparameter Tuning dengan Nested Runs (Optuna + MLflow)

Tujuan

Mencari kombinasi hiperparameter terbaik secara otomatis dengan Optuna, sambil mencatat setiap percobaan (*trial*) sebagai *child run* di bawah satu *parent run* sehingga seluruh percobaan tersusun rapi di MLflow.

1. Instalasi Optuna

```
(.venv) D:\hands-on-mlflow-1>pip install optuna
```

Gambar 15. Instalasi Optuna

```
[notice] A new release of pip is available: 24.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Gambar 16. Notifikasi pembaruan pip setelah instalasi Optuna

2. Membuat skrip train_optuna.py

```
# train_optuna.py
import mlflow
import optuna
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split

# Siapkan data
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_val, y_tr, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Hubungkan ke MLflow
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("breast-cancer-optuna")



# Definisikan fungsi objektif untuk Optuna
def objective(trial):
  # Tentukan rentang parameter yang akan dicari otomatis
  params = {
      "n_estimators": trial.suggest_int("n_estimators", 50, 300),
      "max_depth": trial.suggest_int("max_depth", 3, 15),
      "min_samples_split": trial.suggest_int("min_samples_split", 2, 10),
      "random_state": 42,
  }

  # Buat nested run untuk setiap percobaan (trial) Optuna
  with mlflow.start_run(run_name=f"trial-{trial.number}", nested=True):
    mlflow.log_params(params)

    # Latih model
    model = RandomForestClassifier(**params)
    model.fit(X_tr, y_tr)

    # Evaluasi
    preds = model.predict(X_val)
```

```
    f1 = f1_score(y_val, preds)

    # Log metrik
    mlflow.log_metric("val_f1", f1)

  return f1



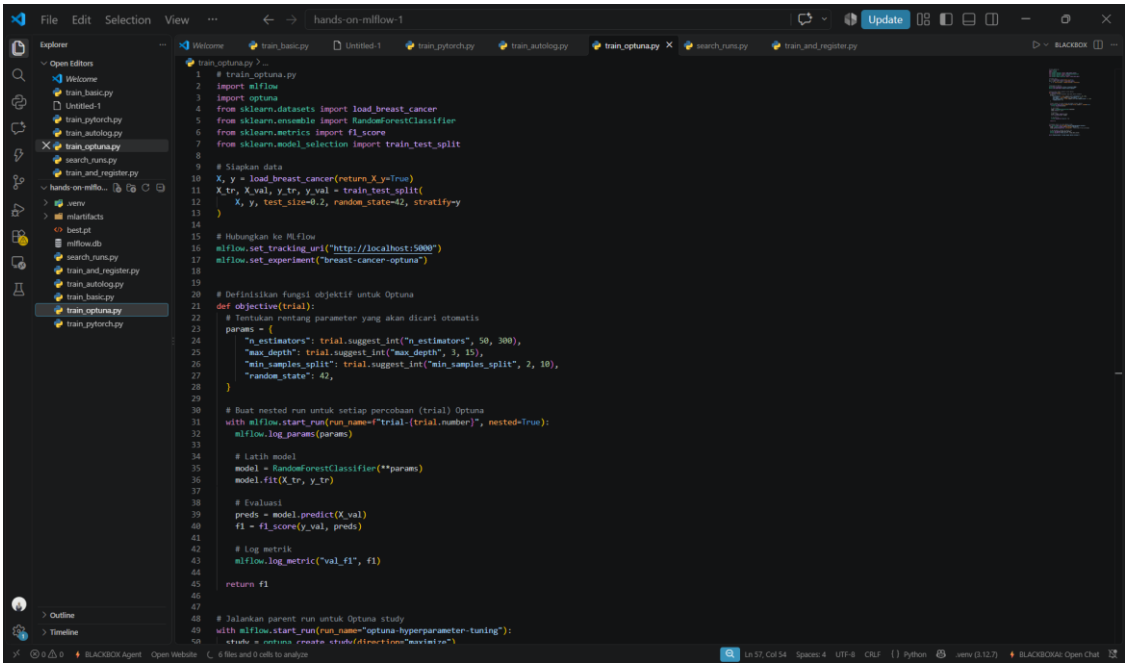
# Jalankan parent run untuk Optuna study
with mlflow.start_run(run_name="optuna-hyperparameter-tuning"):
  study = optuna.create_study(direction="maximize")
  study.optimize(objective, n_trials=5)  # Kita coba 5 kali percobaan

  # Log parameter terbaik ke parent run
  mlflow.log_params(study.best_params)
  mlflow.log_metric("best_val_f1", study.best_value)

print("Hyperparameter tuning dengan Optuna selesai!")
```

Penjelasan kode.

- Fungsi objective(trial) adalah fungsi yang dievaluasi Optuna pada setiap percobaan. Di dalamnya, trial.suggest_int mengusulkan nilai n_estimators (50–300), max_depth (3–15), dan min_samples_split (2–10).
- Setiap percobaan dibungkus mlflow.start_run(..., nested=True) sehingga tercatat sebagai *child run* bernama trial-<nomor> yang berisi parameter dan metrik val_f1 percobaan tersebut.
- Fungsi mengembalikan nilai F1 sebagai skor yang akan dimaksimalkan oleh Optuna.
- optuna.create_study(direction="maximize") membuat *study* yang bertujuan memaksimalkan F1, dan study.optimize(objective, n_trials=5) menjalankan 5 percobaan.
- *Parent run* optuna-hyperparameter-tuning membungkus seluruh *study*. Parameter terbaik (study.best_params) dan skor terbaik (study.best_value) dicatat pada *parent run* ini.



Gambar 17. Kode train_optuna.py di VS Code

3. Menjalankan skrip dan hasilnya

Skrip dijalankan dengan python train_optuna.py. Eksperimen baru breast-cancer-optuna (ID 4) dibuat otomatis, lalu kelima percobaan dijalankan dan dicatat. Hasil tiap percobaan:

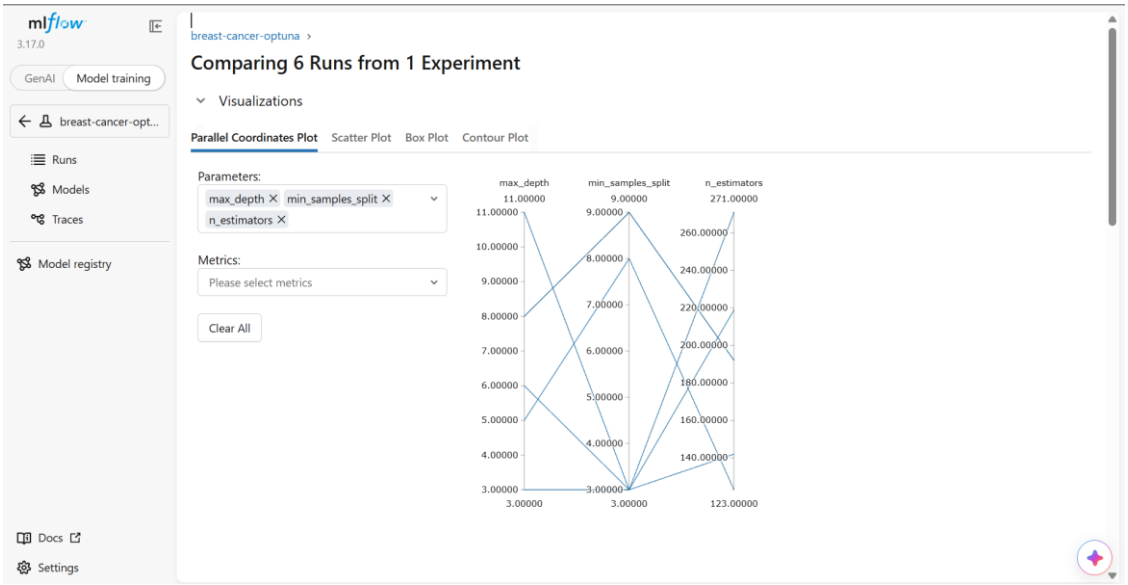
Trial	n_estimators	max_depth	min_samples_split	val_f1
0	192	8	9	0,9655
1	271	6	3	0,9655
2	142	11	3	0,9583
3	123	5	8	0,9655
4	219	3	3	0,9655

Empat dari lima percobaan menghasilkan F1 yang sama (0,9655), sedangkan *trial* 2 sedikit lebih rendah (0,9583). Hal ini menunjukkan bahwa pada data yang relatif kecil ini, model cukup stabil terhadap perubahan hiperparameter pada rentang yang dicoba.

```
(.venv) D:\hands-on-mlflow-1>python train_optuna.py
2026/10/11 03:12:48 INFO mlflow.tracking.fluent: Experiment with name 'breast-canc
er-optuna' does not exist. Creating a new experiment.
[I 2026-10-11 03:12:48,912] A new study created in memory with name: no-name-e4a46
abb-569f-4e21-80cb-b73a73be6687
🏃 View run trial-0 at: http://localhost:5000/#/experiments/4/runs/b078fdc546fc4b4
c83bc0ffd7df5f8d9
🧪 View experiment at: http://localhost:5000/#/experiments/4
[I 2026-10-11 03:12:49,588] Trial 0 finished with value: 0.9655172413793104 and pa
rameters: {'n_estimators': 192, 'max_depth': 8, 'min_samples_split': 9}. Best is t
rial 0 with value: 0.9655172413793104.
🏃 View run trial-1 at: http://localhost:5000/#/experiments/4/runs/30aac3c6c41048d
494c87714312cbe48
🧪 View experiment at: http://localhost:5000/#/experiments/4
[I 2026-10-11 03:12:50,448] Trial 1 finished with value: 0.9655172413793104 and pa
rameters: {'n_estimators': 271, 'max_depth': 6, 'min_samples_split': 3}. Best is t
rial 0 with value: 0.9655172413793104.
🏃 View run trial-2 at: http://localhost:5000/#/experiments/4/runs/6ac2bf27396c4de
988a2f97362691839
🧪 View experiment at: http://localhost:5000/#/experiments/4
[I 2026-10-11 03:12:51,036] Trial 2 finished with value: 0.9583333333333334 and pa
rameters: {'n_estimators': 142, 'max_depth': 11, 'min_samples_split': 3}. Best is
trial 0 with value: 0.9655172413793104.
🏃 View run trial-3 at: http://localhost:5000/#/experiments/4/runs/348699fdc27c478
896ff6db19da858c3
🧪 View experiment at: http://localhost:5000/#/experiments/4
[I 2026-10-11 03:12:51,586] Trial 3 finished with value: 0.9655172413793104 and pa
rameters: {'n_estimators': 123, 'max_depth': 5, 'min_samples_split': 8}. Best is t
rial 0 with value: 0.9655172413793104.
🏃 View run trial-4 at: http://localhost:5000/#/experiments/4/runs/d35c0b99c465498
a807d6793c82c14b2
🧪 View experiment at: http://localhost:5000/#/experiments/4
[I 2026-10-11 03:12:52,273] Trial 4 finished with value: 0.9655172413793104 and pa
rameters: {'n_estimators': 219, 'max_depth': 3, 'min_samples_split': 3}. Best is t
rial 0 with value: 0.9655172413793104.
🏃 View run optuna-hyperparameter-tuning at: http://localhost:5000/#/experiments/4
/runs/ae730919f7494bab86d574f663ef6406
🧪 View experiment at: http://localhost:5000/#/experiments/4
Hyperparameter tuning dengan Optuna selesai!
```

Gambar 18. Hasil menjalankan train_optuna.py (5 trial)

Pada MLflow UI, eksperimen breast-cancer-optuna menampilkan satu *parent run* dengan lima *child run* di dalamnya yang dapat dipilih dan dibandingkan menggunakan *parallel coordinates plot*.



Gambar 19. Parent run dengan child run hasil Optuna di MLflow UI

E. Membandingkan & Mencari Run secara Programatik

Tujuan

Mencari *run* terbaik secara otomatis lewat kode, bukan hanya memeriksa satu per satu di UI, serta membandingkan beberapa *run* secara visual di MLflow UI.

1. Pencarian run dengan search_runs.py

```
# search_runs.py
import mlflow

# Melakukan pencarian run secara programatik berdasarkan filter dan urutan metrik
runs = mlflow.search_runs(
    experiment_names=["breast-cancer-optuna"],
    filter_string="metrics.val_f1 > 0.90",
    order_by=["metrics.val_f1 DESC"],
    max_results=10,
)

# Menampilkan kolom penting dari hasil pencarian runs
print(
    runs[["run_id", "params.n_estimators", "params.max_depth", "metrics.val_f1"]]
)

# Mengambil run_id terbaik dari baris pertama (index 0)
if not runs.empty:
  best_run_id = runs.iloc[0]["run_id"]
  print(f"\nRun ID terbaik ditemukan: {best_run_id}")
else:
  print("Tidak ada run yang sesuai dengan filter.")
```

Penjelasan kode.

- mlflow.search_runs mengambil daftar *run* dari eksperimen breast-cancer-optuna dalam bentuk *DataFrame* pandas.
- filter_string="metrics.val_f1 > 0.90" hanya menyertakan *run* dengan F1 di atas 0,90, order_by mengurutkan dari F1 tertinggi, dan max_results=10 membatasi jumlah hasil.
- Kolom penting (ID run, n_estimators, max_depth, val_f1) ditampilkan, lalu ID *run* pada baris pertama disimpan sebagai best_run_id. Jika tidak ada *run* yang memenuhi filter, program menampilkan pesan bahwa hasil tidak ditemukan.

2. Hasil pencarian

Keluaran menampilkan lima *run* (kelima *child run* dari Optuna) yang memenuhi filter, terurut dari F1 tertinggi. *Run* pertama berisi ID d35c0b99c465498a807d6793c82c14b2 dan menjadi best_run_id. ID ini sesuai dengan trial-4 pada tahap D (n_estimators=219, max_depth=3, min_samples_split=3). Karena ada beberapa *run* dengan F1 yang sama, urutan di antara *run* tersebut ditentukan oleh MLflow, dan parameter dari trial-4 inilah yang dipakai pada tahap F.

```
(.venv) D:\hands-on-mlflow-1>python search_runs.py
                            run_id  ... metrics.val_f1
0  d35c0b99c465498a807d6793c82c14b2  ...        0.965517
1  348699fdc27c478896ff6db19da858c3  ...        0.965517
2  30aac3c6c41048d494c87714312cbe48  ...        0.965517
3  b078fdc546fc4b4c83bc0ffd7df5f8d9  ...        0.965517
4  6ac2bf27396c4de988a2f97362691839  ...        0.958333

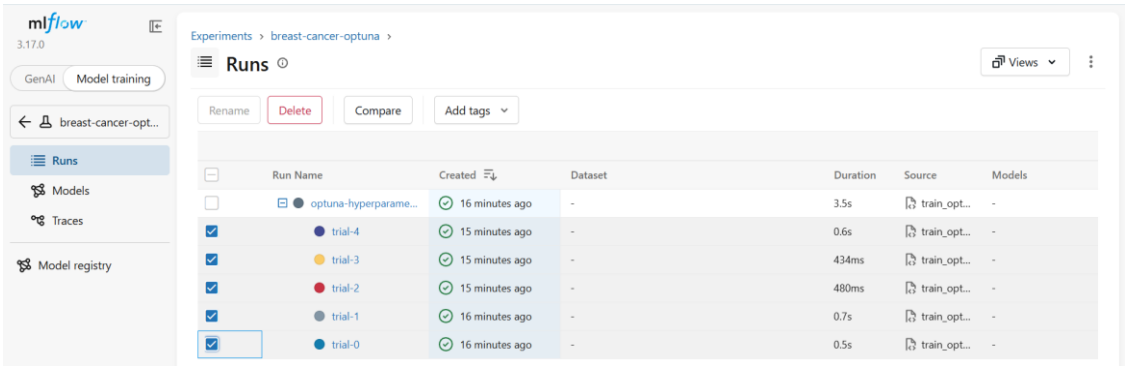
[5 rows x 4 columns]

Run ID terbaik ditemukan: d35c0b99c465498a807d6793c82c14b2
```

Gambar 20. Hasil menjalankan search_runs.py

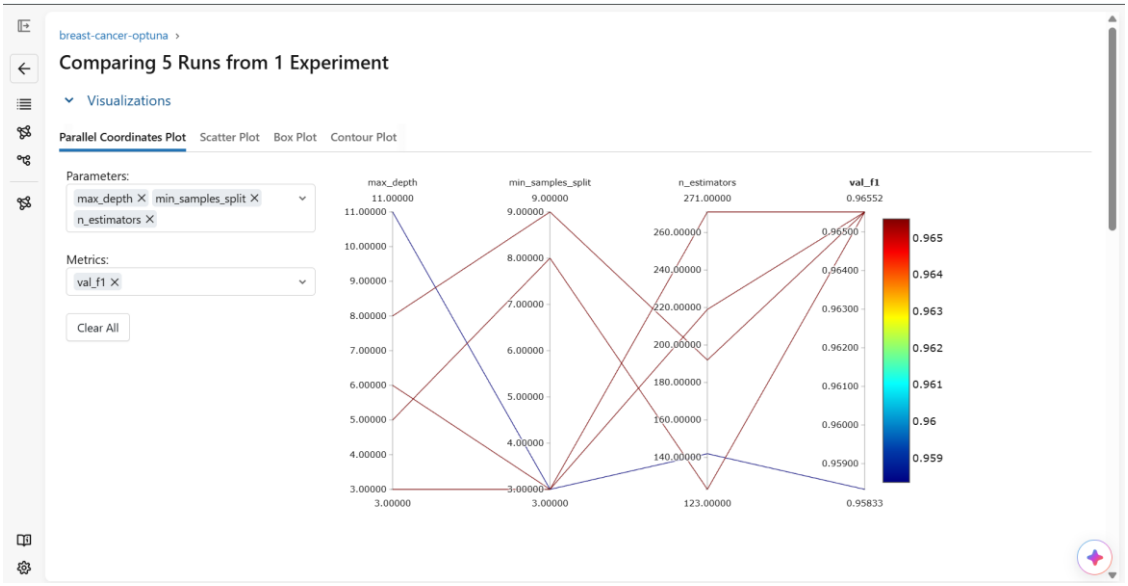
3. Membandingkan run di MLflow UI

Pada MLflow UI (http://localhost:5000), beberapa *run* dipilih lalu menu **Compare** digunakan. Dari sini tersedia beberapa jenis visualisasi:



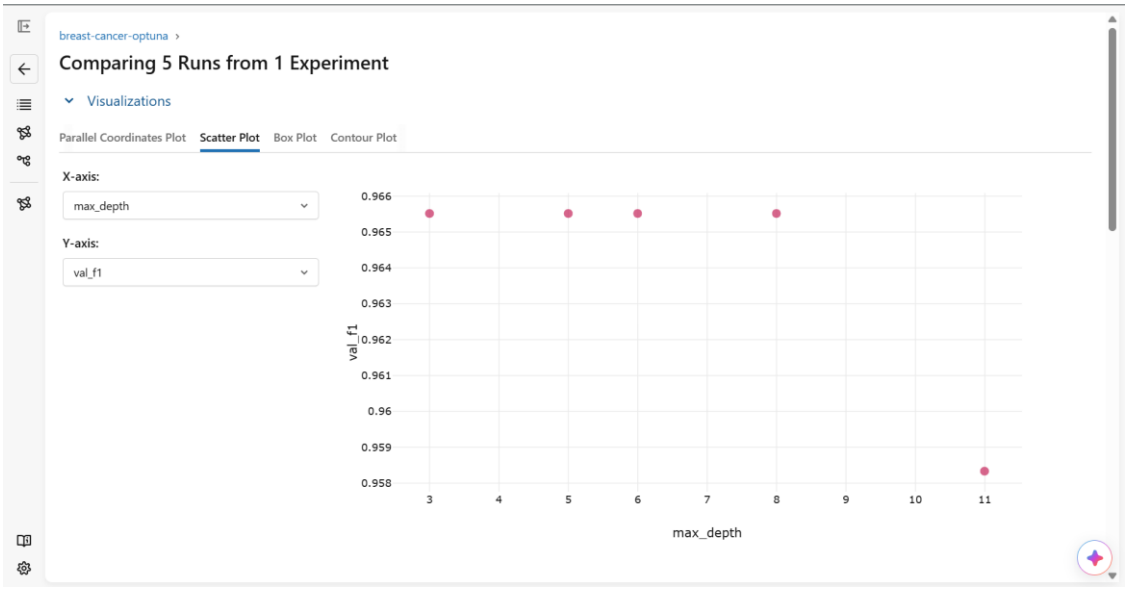
Gambar 21. Memilih beberapa run untuk dibandingkan (Compare)

- **Parallel coordinates plot**: setiap garis mewakili satu *run* yang melintasi nilai parameter hingga metrik, sehingga terlihat kombinasi parameter mana yang menghasilkan metrik tinggi.



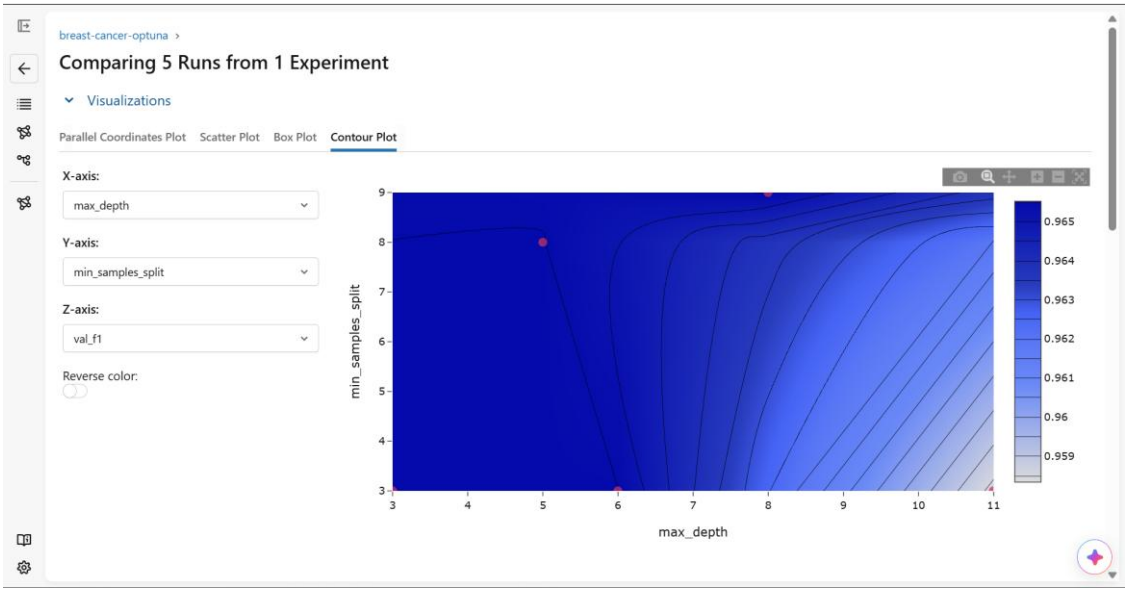
Gambar 22. Parallel coordinates plot

- **Scatter plot**: menampilkan hubungan antara satu parameter (misalnya max_depth) dan metrik val_f1.



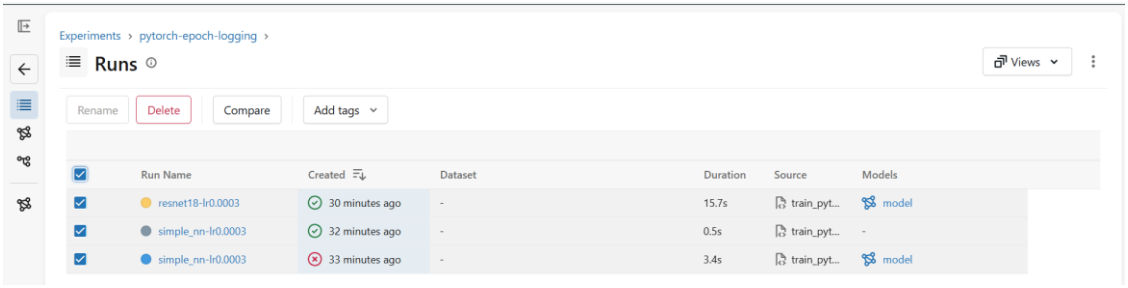
Gambar 23. Scatter plot

- **Contour plot**: menampilkan hubungan dua parameter terhadap metrik dalam bentuk peta warna.

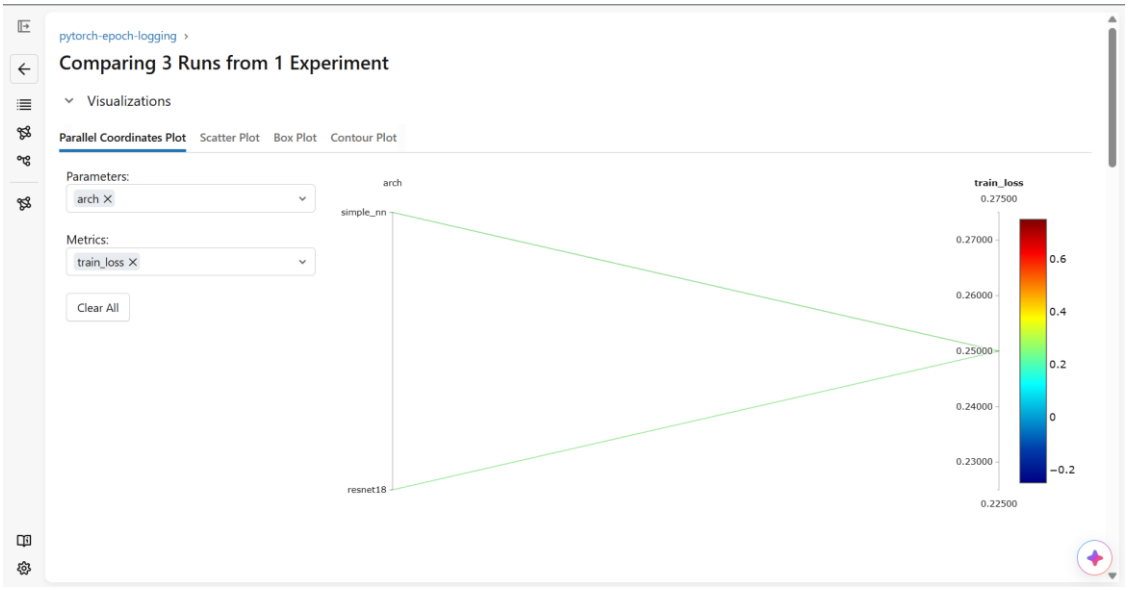


Gambar 24. Contour plot

- **Overlay learning curves**: menumpuk kurva metrik per *epoch* dari beberapa *run* dalam satu grafik. Fitur ini berguna pada eksperimen yang mencatat metrik per *epoch* atau *step*, seperti eksperimen PyTorch pada tahap B (pytorch-epoch-logging). Karena fungsi pelatihan pada tahap B berupa simulasi yang selalu mengembalikan nilai tetap, kurva metriknya secara logika akan berbentuk datar.

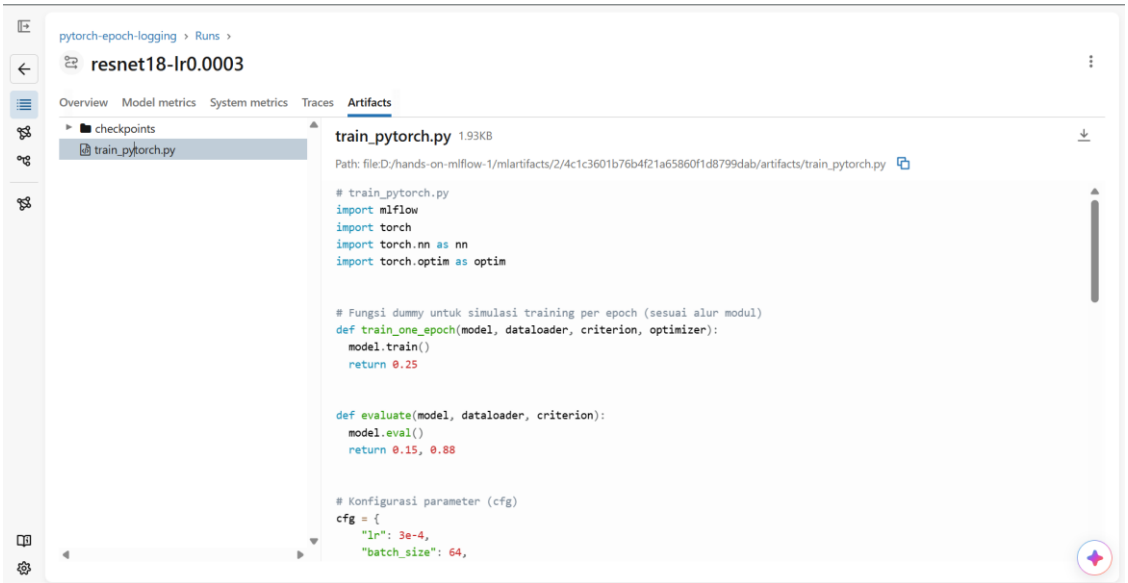


Gambar 25. Memilih run dari eksperimen pytorch-epoch-logging



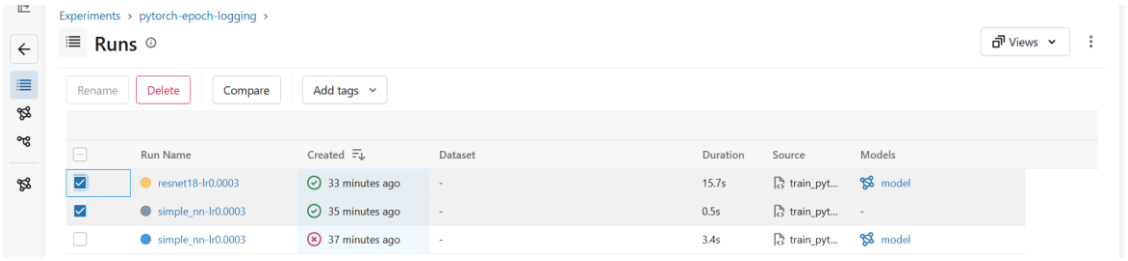
Gambar 26. Perbandingan metrik per epoch antar run

- **Artifacts**: pada halaman sebuah *run*, tab *Artifacts* menampilkan file yang tersimpan, seperti model, *checkpoint*, dan skrip yang diunggah.

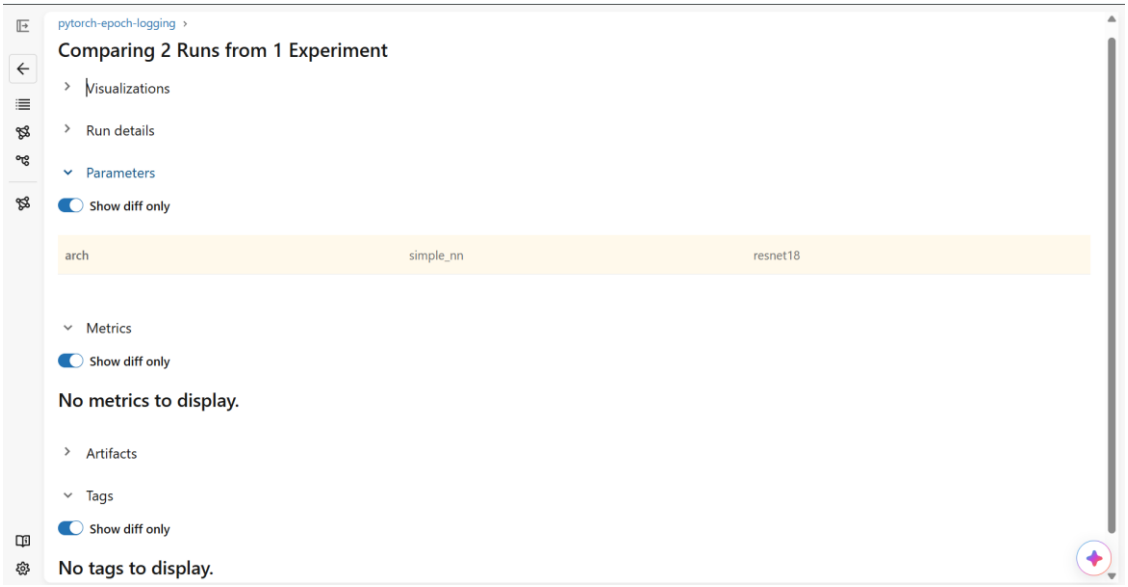


Gambar 27. Tab Artifacts pada sebuah run

- **Diff parameter**: membandingkan perbedaan parameter secara mendetail antara dua *run* dengan opsi *Show diff only*, sehingga hanya parameter yang berbeda yang ditampilkan.



Gambar 28. Memilih dua run untuk dibandingkan parameternya



Gambar 29. Tampilan diff parameter antara dua run

F. Model Registry: Register → Alias → Load

Tujuan

Mengelola model hasil terbaik secara terstruktur: mendaftarkannya ke *Model Registry* agar memiliki nama dan versi, memberinya anotasi serta alias, lalu memuatnya kembali lewat alias tanpa perlu menuliskan nomor versi.

1. Membuat skrip train_and_register.py

Skrip ini memakai hiperparameter terbaik dari eksperimen Optuna (trial-4), yaitu n_estimators=219, max_depth=3, dan min_samples_split=3.

```
# train_and_register.py
import mlflow
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# 1. Hubungkan ke server MLflow lokal
mlflow.set_tracking_uri("http://localhost:5000")
```

```
mlflow.set_experiment("breast-cancer-optuna")

# 2. Siapkan data
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_val, y_tr, y_val = train_test_split(
    X, y, test  size=0.2, random  state=42, stratify=y
)

# 3. Mulai run baru untuk melatih dan mendaftarkan model secara langsung
with mlflow.start_run(run_name="model-registry-run") as run:
  params = {"n_estimators": 219, "max_depth": 3, "min_samples_split": 3}
  mlflow.log_params(params)

  # Latih model
  model = RandomForestClassifier(**params, random_state=42)
  model.fit(X_tr, y_tr)

  # Log model menggunakan serialization_format="cloudpickle" agar lolos dari validasi skops
  mlflow.sklearn.log_model(
     model, name="model", serialization_format="cloudpickle"
  )

  # 4. Daftarkan model ke Model Registry
  model_uri = f"runs:/{run.info.run_id}/model"
  name = "breast-cancer-clf"

  print("Melakukan registrasi model...")
  mv = mlflow.register_model(model_uri, name)
  print(f"Model berhasil didaftarkan! Versi model: {mv.version}")

  # 5. Berikan anotasi, tag, dan alias (@challenger & @champion)
  client = mlflow.MlflowClient()
  client.update_model_version(
     name, mv.version, description="RF depth=3, hasil tuning Optuna"
  )
  client.set_model_version_tag(name, mv.version, "validation_status", "passed")
  client.set_registered_model_alias(name, "challenger", mv.version)
  client.set_registered_model_alias(name, "champion", mv.version)
  print("Alias '@challenger' dan '@champion' berhasil disematkan!")

  # 6. Uji muat model via alias
  loaded_model = mlflow.pyfunc.load_model(f"models:/{name}@champion")
  print("Model berhasil dimuat ulang menggunakan alias '@champion'!")
```

Penjelasan kode.

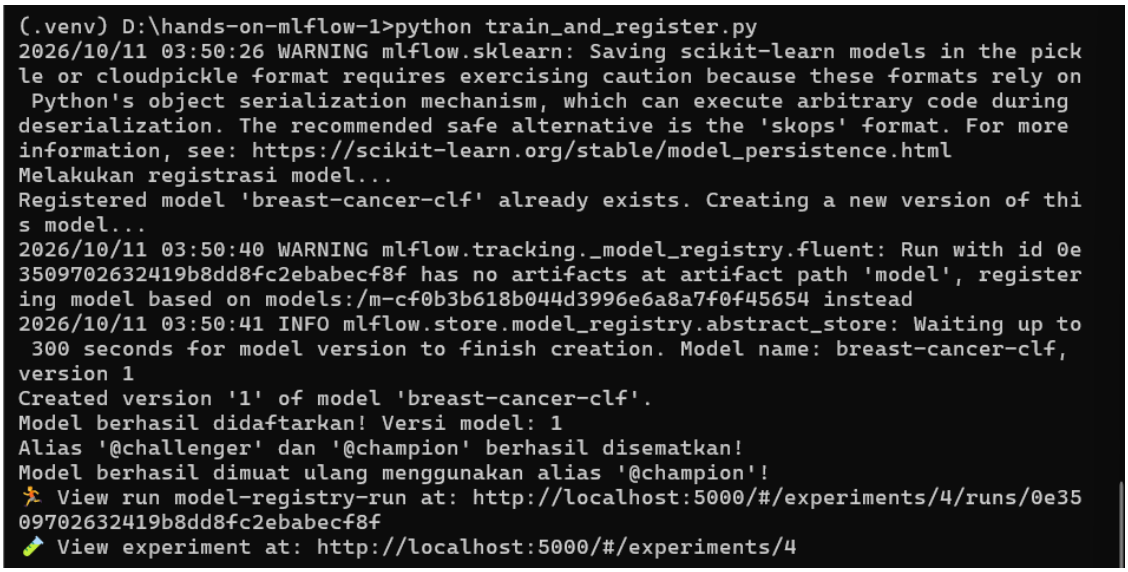
- Skrip membuka *run* baru model-registry-run pada eksperimen breast-cancer-optuna, mencatat parameter, lalu melatih ulang model dengan parameter terbaik.
- Model disimpan dengan serialization_format="cloudpickle", sebagaimana dicatat pada komentar kode, agar lolos dari validasi skops.
- mlflow.register_model mendaftarkan model dari URI runs:/<run_id>/model ke *Model Registry* dengan nama breast-cancer-clf. Pendaftaran pertama menghasilkan versi 1.
- MlflowClient dipakai untuk menambahkan deskripsi versi model, tag validation_status=passed, serta dua alias: @challenger (calon model produksi) dan @champion (model yang dipakai produksi). Alias berfungsi sebagai penunjuk ke sebuah

versi, sehingga aplikasi cukup memuat @champion tanpa mengubah kode ketika versinya berganti.

- mlflow.pyfunc.load_model("models:/breast-cancer-clf@champion") menguji bahwa model dapat dimuat kembali melalui alias.

2. Menjalankan skrip dan hasilnya

Skrip dijalankan dengan python train_and_register.py. Keluaran menunjukkan bahwa model breast-cancer-clf berhasil dibuat sebagai versi 1, alias @challenger dan @champion berhasil disematkan, dan model berhasil dimuat ulang melalui alias @champion. Peringatan yang muncul berkaitan dengan penyimpanan model berformat *cloudpickle* (sama seperti pada tahap B) dan penyesuaian lokasi *artifact* pada MLflow 3, yang tidak menghalangi proses registrasi.



Gambar 30. Hasil menjalankan train_and_register.py

Kesimpulan

Melalui praktik Hands-on 1, telah dilakukan seluruh alur dasar pengelolaan eksperimen dengan MLflow. Parameter, metrik, tag, dan model dicatat secara terpusat (tahap A dan B), pencatatan dapat disederhanakan dengan *autologging* (tahap C), pencarian hiperparameter dapat tersusun rapi lewat *nested runs* (tahap D), *run* terbaik dapat dicari dan dibandingkan secara programatik maupun visual (tahap E), dan model terbaik dapat dikelola lewat versi serta alias pada *Model Registry* (tahap F). Dengan alur ini, setiap percobaan dapat ditelusuri dan direproduksi, yang merupakan prinsip penting dalam praktik MLOps.